Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: Funcții, metode și importuri pe web

A efectuat : st. gr.SI – 264: Railean Adrian

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

1. Sarcina lucrării

Scopul acestui laborator este însușirea și practicarea conceptelor fundamentale privind comunicațiile web în Python, utilizând module din biblioteca standard și biblioteci externe. Lucrarea este structurată în 5 părți principale, având ca obiective:

1. Înțelegerea și utilizarea diferitelor stiluri de import (import direct, import specific, alias-uri) și gestionarea dependențelor externe prin pip.

2. Analiza obiectelor de răspuns HTTP, diferențierea dintre atribute și metode, gestionarea antetelor, cookie-urilor, sesiunilor și redirecționărilor.

3. Dezvoltarea de funcții proprii modulare, cu parametri, adnotări de tip, docstring-uri, valori implicite și tratarea erorilor de rețea.

4. Combinarea modulelor din biblioteca standard (urllib.parse, re, html.parser, hashlib, json, socket, ssl) pentru parsare, securitate și operații de rețea.

5. Structurarea codului într-un modul propriu reutilizabil (webtools.py), construirea uni interfețe în linia de comandă (main.py) și generarea rapoartelor finale în formatele CSV și JSON.

2. Executarea lucrării

2.1 Partea 1: Importuri și biblioteca standard

A fost creat fișierul part1_imports.py în care s-au testat importurile modulului requests și urllib.request, măsurarea timpului de răspuns cu time.perf_counter() și gestionarea erorilor de import prin blocuri try/except.



Figura 2.1 – Rularea modulului part1_imports.py și verificarea stilurilor de import

2.2 Partea 2: Metode și atribute ale obiectelor web

În fișierul part2_methods.py s-au analizat atributele (status_code, ok, url) și metodele (raise_for_status, json, headers.get) ale obiectului Response, s-a testat urmărirea redirecționărilor și crearea unei sesiuni cu User-Agent personalizat.

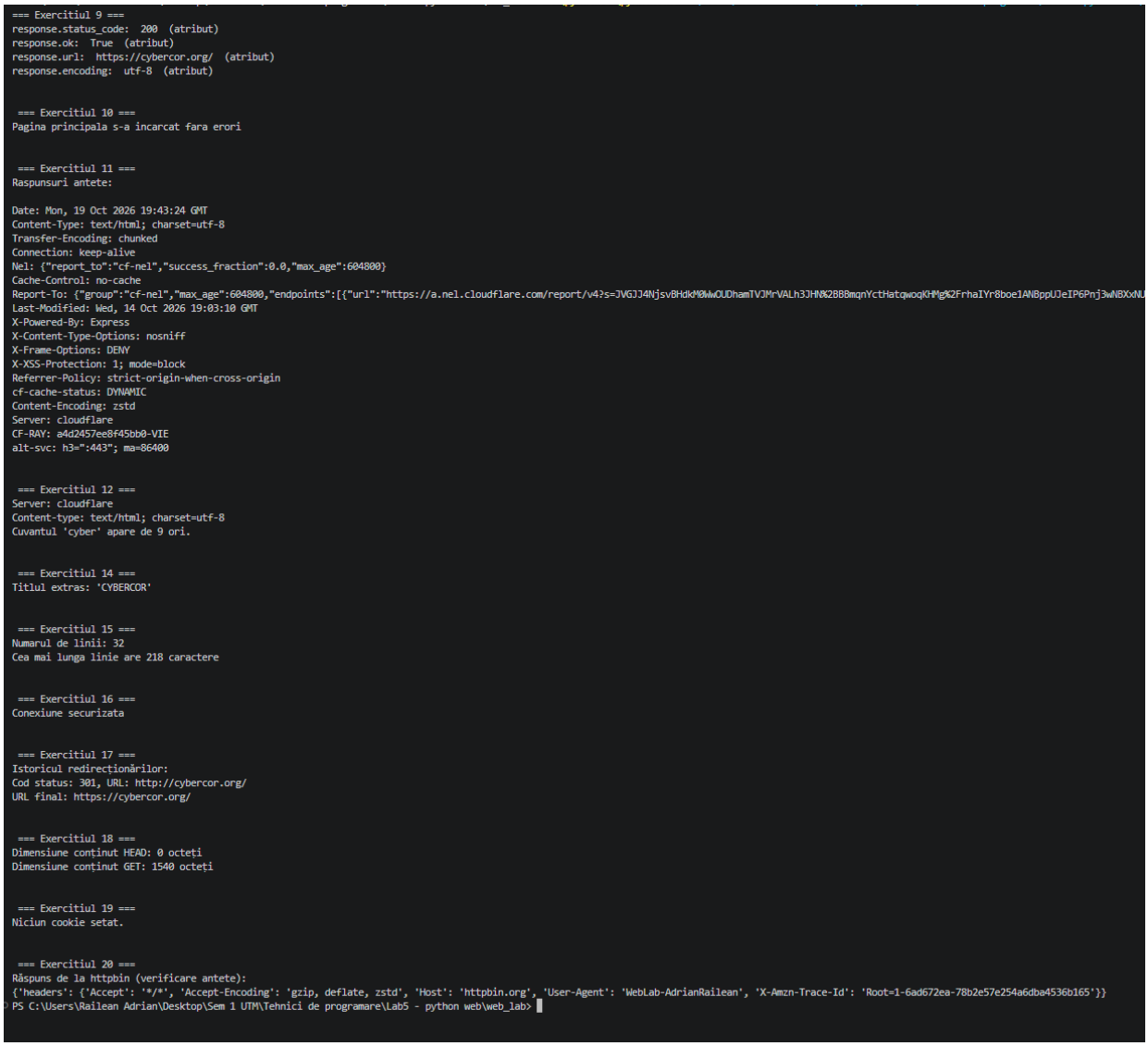


Figura 2.2 – Rezultatele executării part2_methods.py și inspectarea antetelor HTTP

2.3 Partea 3: Scrierea propriilor funcții

În part3_functions.py s-au definit funcții modulare pentru descărcarea paginilor, extragerea titlului, evaluarea antetelor de securitate (Strict-Transport-Security, CSP, etc.), parsarea fișierului robots.txt și funcții cu argumente variabile (*urls, **details).

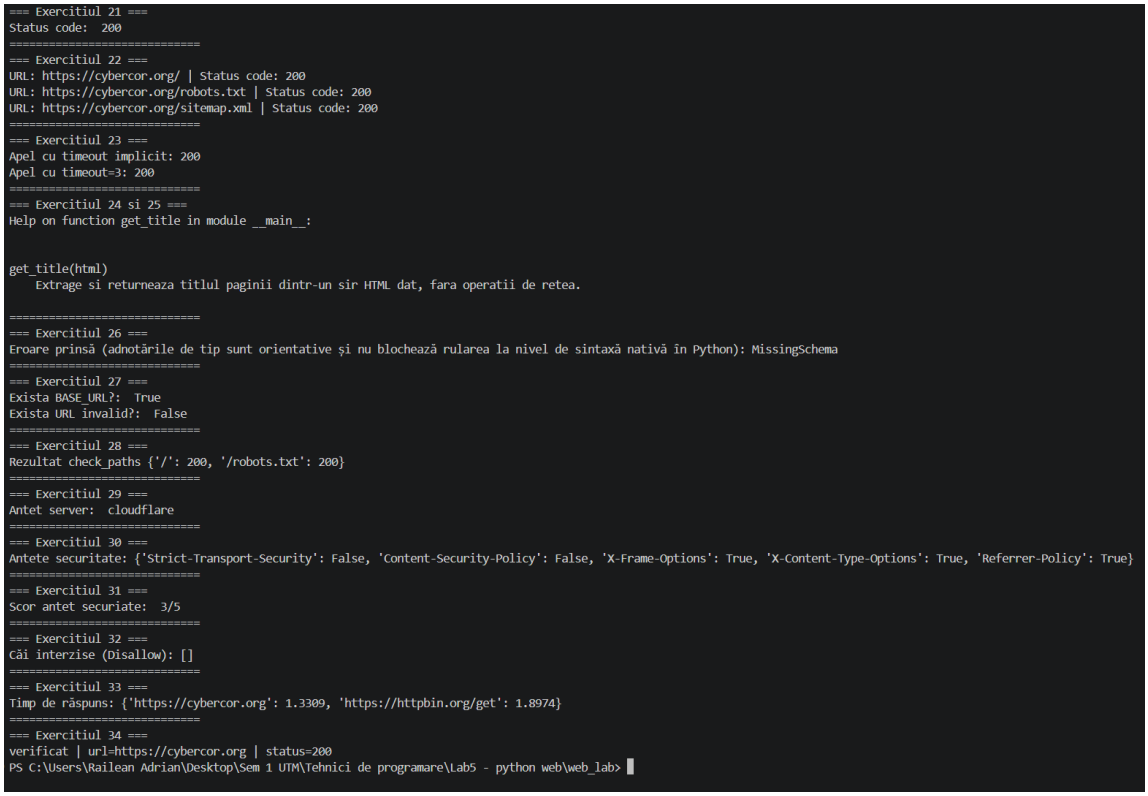


Figura 2.3 – Rularea funcțiilor custom din part3_functions.py și scorul antetelor de securitate

2.4 Partea 4: Module din biblioteca standard

Fișierul part4_modules.py combină module avansate pentru descompunerea URL-urilor, extragerea legăturilor prin expresii regulate, crearea unui parser HTML custom (ImageFinder), calcularea amprentei SHA-256, rezolvarea DNS și verificarea expirării certificatului SSL.

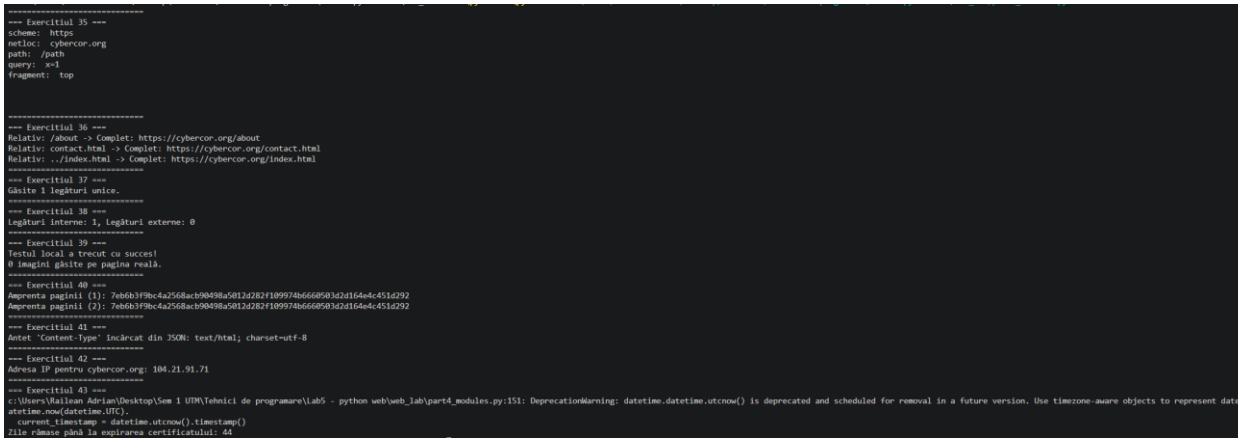


Figura 2.4 – Rularea part4_modules.py: parser HTML, rezolvare DNS și verificare certificat SSL

2.5 Partea 5: Modulul webtools.py și scriptul main.py

Toate funcțiile reutilizabile au fost mutate în modulul webtools.py, dotat cu blocul de autotest if __name__ == '__main__':. Scriptul main.py folosește argparse pentru preluarea URL-ului din linia de comandă și apelează webtools.site_report().



Figura 2.5 – Executarea autotestului din modulul webtools.py

```
Pornire analiză pentru: https://cybercor.org

Raportul CSV a fost salvat cu succes în report.csv

=== Raport site: https://cybercor.org ===
Cod de stare:      200
Titlu:             CYBERCOR
Adresă IP:         104.21.91.71
Redirecționări:    301 -> https://cybercor.org/
Scor securitate:   3/5
Certificat:        44 de zile rămase
Legături:          1 interne, 0 externe
Căi interzise:     Niciuna
Salvat în report.json
```

Figura 2.6 – Executarea comenzii python main.py https://cybercor.org în terminal

2.6 Generarea rapoartelor de ieșire

Executarea scriptului principal a generat fișierele de ieșire report.csv (conținând statusul căilor verificate) și report.json (raportul structurat al domeniului).

report.csv

```
1  path,status,checked_at
2  /,200,2026-10-19T22:48:56.596359
3  /robots.txt,200,2026-10-19T22:48:56.596359
4  /sitemap.xml,200,2026-10-19T22:48:56.596359
5
```

Figura 2.7 – Conținutul fișierului de ieșire report.csv

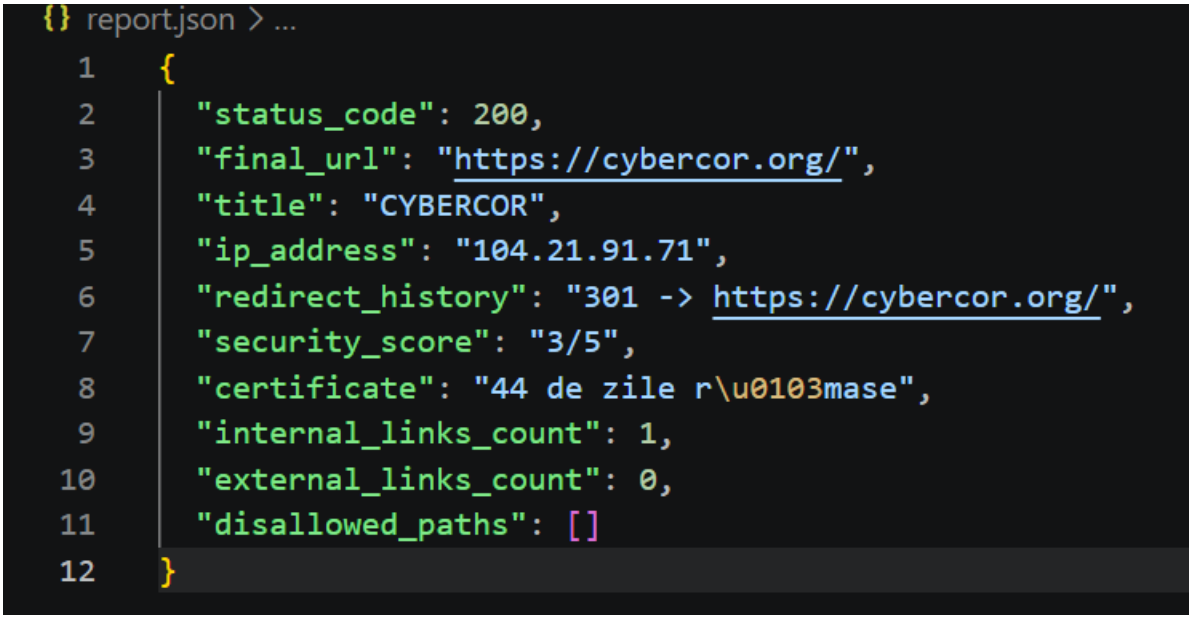


Figura 2.8 – Conținutul fișierului structurat report.json

3. Concluzii

În cadrul acestui laborator am aprofundat conceptele fundamentale legate de comunicațiile prin protocolul HTTP în limbajul Python. Am învățat să fac distincția clară între module din biblioteca standard (cum ar fi urllib, socket, ssl, json) și pachete externe instalabile prin pip (cum este requests).

De asemenea, am exersat organizarea codului conform celor mai bune practici: separarea logicii de rețea de parsarea datelor, utilizarea adnotărilor de tip și a docstring-urilor, gestionarea excepțiilor de rețea pentru a preveni oprirea neașteptată a programului, și împărțirea aplicației într-un modul reutilizabil (webtools.py) și un punct de intrare CLI (main.py). Generarea rapoartelor finale în format CSV și JSON demonstrează capacitatea de a automatiza colectarea și structurarea datelor de pe site-uri web reale.

4. Bibliografie

1. Documentation Python 3 - Standard Library: https://docs.python.org/3/library/

2. Requests: HTTP for Humans Documentation: https://requests.readthedocs.io/

3. HTTPbin Service for HTTP Testing: https://httpbin.org/

4. Cybercor Center Official Website: https://cybercor.org/

5. Anexe

Anexa A. Codul fișierului part1_imports.py

```
# Laborator: funcții, metode și importuri pe web
# Student: Adrian Railean

import requests
import urllib.request
import time

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

# --- Exercițiul 1. Primul import ---
print("--- Exercițiul 1 ---")
print("Versiune requests:", requests.__version__)
# 'requests' este o bibliotecă externă (necesită pip install), iar 'urllib'
face parte din biblioteca standard.

# --- Exercițiul 2. Două stiluri de import ---
print("\n--- Exercițiul 2 ---")
response1 = requests.get(BASE_URL, timeout=TIMEOUT)
print("Stil 1 - Status code:", response1.status_code)

time.sleep(1)
from requests import get
response2 = get(BASE_URL, timeout=TIMEOUT)
print("Stil 2 - Status code:", response2.status_code)

# --- Exercițiul 3. Alias-uri ---
print("\n--- Exercițiul 3 ---")
import requests as rq
time.sleep(1)
response_alias = rq.get(BASE_URL, timeout=TIMEOUT)
print("Alias rq - Status code:", response_alias.status_code)

# --- Exercițiul 4. Doar biblioteca standard ---
print("\n--- Exercițiul 4 ---")
time.sleep(1)
req_std = urllib.request.urlopen(BASE_URL)
print("Urllib status:", req_std.status)
body_snippet = req_std.read(200).decode("utf-8")
print("Primele 200 caractere:\n", body_snippet)

# --- Exercițiul 5. Priviți în interiorul unui modul ---
print("\n--- Exercițiul 5 ---")
# requests.get (funcție), requests.Session (clasă), requests.exceptions (sub-
modul)

# --- Exercițiul 6. Citiți documentația ---
print("\n--- Exercițiul 6 ---")
time.sleep(1)
res_timeout = requests.get(BASE_URL, timeout=TIMEOUT)
print("Cerere cu timeout reușită:", res_timeout.status_code)

# --- Exercițiul 7. Cronometrarea unei cereri ---
print("\n--- Exercițiul 7 ---")
```

```
time.sleep(1)
start_time = time.perf_counter()
res_chrono = requests.get(BASE_URL, timeout=TIMEOUT)
end_time = time.perf_counter()
print(f"Timp măsurat cu perf_counter: {end_time - start_time:.4f} secunde")
print(f"Timp raportat de response.elapsed: {res_chrono.elapsed.total_seconds()}
secunde")

# --- Exercițiul 8. Când un import eșuează ---
print("\n--- Exercițiul 8 ---")
try:
    import bs4
    print("Modulul bs4 este instalat.")
except ImportError:
    print("Instalați modulul cu: pip install beautifulsoup4")
```

Anexa B. Codul fișierului part2_methods.py

```
# Laborator: funcții, metode și importuri pe web
# Student: Adrian Railean

import requests
import time

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

# --- Exercițiul 9. Elementele de bază ale răspunsului ---
print("--- Exercițiul 9 ---")
response = requests.get(BASE_URL, timeout=TIMEOUT)
print(f"response.status_code: {response.status_code} (atribut)")
print(f"response.ok: {response.ok} (atribut)")
print(f"response.url: {response.url} (atribut)")
print(f"response.encoding: {response.encoding} (atribut)")

# --- Exercițiul 10. Tratarea erorilor cu o metodă ---
print("\n--- Exercițiul 10 ---")
time.sleep(1)
try:
    response.raise_for_status()
    print("Pagina principală s-a încărcat cu succes.")
    time.sleep(1)
    bad_response = requests.get(BASE_URL + "/this-page-does-not-exist",
timeout=TIMEOUT)
    bad_response.raise_for_status()
except requests.HTTPError as e:
    print(f"Eroare capturată: {e}")

# --- Exercițiul 11. Toate antetele ---
print("\n--- Exercițiul 11 ---")
for header_name, header_value in response.headers.items():
    print(f"{header_name}: {header_value}")

# --- Exercițiul 12. Metode de dicționar ---
print("\n--- Exercițiul 12 ---")
print(f"Server: {response.headers.get('Server', 'lipsește')}")
print(f"Content-Type: {response.headers.get('Content-Type', 'lipsește')}")

# --- Exercițiul 13. Înlănțuirea metodelor de șir ---
```

```
print("\n--- Exercițiul 13 ---")
print(f"Cuvântul 'cyber' apare de {response.text.lower().count('cyber')} ori.")

# --- Exercițiul 14. Găsirea titlului ---
print("\n--- Exercițiul 14 ---")
start_tag, end_tag = "<title>", "</title>"
start_idx = response.text.find(start_tag)
if start_idx != -1:
    start_pos = start_idx + len(start_tag)
    end_pos = response.text.find(end_tag, start_pos)
    print(f"Titlul extras: '{response.text[start_pos:end_pos].strip()}'")

# --- Exercițiul 15. Numărarea liniilor ---
print("\n--- Exercițiul 15 ---")
lines = response.text.splitlines()
print(f"Număr linii: {len(lines)}, cea mai lungă linie: {len(max(lines,
key=len))} caractere")

# --- Exercițiul 16. Verificarea HTTPS ---
print("\n--- Exercițiul 16 ---")
print("Conexiune securizată" if response.url.startswith("https://") else
"Conexiune nesecurizată")

# --- Exercițiul 17. Urmărirea redirecționărilor ---
print("\n--- Exercițiul 17 ---")
time.sleep(1)
response_http = requests.get("http://cybercor.org", timeout=TIMEOUT)
for history_resp in response_http.history:
    print(f"Cod status: {history_resp.status_code}, URL: {history_resp.url}")
print(f"URL final: {response_http.url}")

# --- Exercițiul 18. HEAD versus GET ---
print("\n--- Exercițiul 18 ---")
time.sleep(1)
res_head = requests.head(BASE_URL, timeout=TIMEOUT)
time.sleep(1)
res_get = requests.get(BASE_URL, timeout=TIMEOUT)
print(f"Dimensiune conținut HEAD: {len(res_head.content)}, GET:
{len(res_get.content)}")

# --- Exercițiul 19. Cookie-uri ---
print("\n--- Exercițiul 19 ---")
if response.cookies:
    for cookie in response.cookies:
        print(f"Cookie: {cookie.name}, Secure: {cookie.secure}")
else:
    print("Niciun cookie setat.")

# --- Exercițiul 20. Sesiuni ---
print("\n--- Exercițiul 20 ---")
time.sleep(1)
session = requests.Session()
session.headers.update({"User-Agent": "WebLab-AdrianRailean"})
res_session = session.get(ECHO_URL + "/headers", timeout=TIMEOUT)
print("Sesiune User-Agent:", res_session.json())
```

Anexa C. Codul fișierului part3_functions.py

```
# Laborator: funcții, metode și importuri pe web
# Student: Adrian Railean
```

```
import requests
import time

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

def fetch(url: str, timeout: int = TIMEOUT):
    """Descarcă o pagină web și returnează obiectul răspuns."""
    return requests.get(url, timeout=timeout)

def get_status(url: str) -> int:
    """Returnează codul de stare HTTP pentru adresa url."""
    response = requests.get(url, timeout=TIMEOUT)
    return response.status_code

def get_title(html: str) -> str:
    """Extrage și returnează titlul paginii dintr-un șir HTML dat, fără
operații de rețea."""
    start_tag, end_tag = "<title>", "</title>"
    start_idx = html.find(start_tag)
    if start_idx != -1:
        start_pos = start_idx + len(start_tag)
        end_pos = html.find(end_tag, start_pos)
        if end_pos != -1:
            return html[start_pos:end_pos].strip()
    return "Titlu negăsit"

def page_exists(url: str) -> bool:
    """Verifică dacă un URL există și poate fi accesat, returnând True sau
False."""
    try:
        response = requests.get(url, timeout=TIMEOUT)
        return response.ok
    except requests.RequestException:
        return False

def check_paths(base: str, paths: list) -> dict:
    """Verifică o listă de căi pe un domeniu și returnează un dicționar {path:
status_code}."""
    results = {}
    for path in paths:
        try:
            res = requests.get(base + path, timeout=TIMEOUT)
            results[path] = res.status_code
        except requests.RequestException:
            results[path] = None
        time.sleep(1)
    return results

def get_header(url: str, name: str, default: str = "lipsește") -> str:
    """Returnează valoarea unui singur antet HTTP specificat."""
    try:
        res = requests.get(url, timeout=TIMEOUT)
        return res.headers.get(name, default)
    except requests.RequestException:
        return default

def security_headers(url: str) -> dict:
```

```
    """Verifică prezența a 5 antete de securitate și returnează un dicționar
{antet: True/False}."""
    headers_to_check = [
        "Strict-Transport-Security", "Content-Security-Policy",
        "X-Frame-Options", "X-Content-Type-Options", "Referrer-Policy"
    ]
    try:
        res = requests.get(url, timeout=TIMEOUT)
        return {h: h in res.headers for h in headers_to_check}
    except requests.RequestException:
        return {h: False for h in headers_to_check}

def score_headers(results: dict) -> str:
    """Calculează un scor sub formă de șir (ex: '3/5') bazat pe dicționarul de
antete."""
    count = sum(1 for val in results.values() if val)
    return f"{count}/{len(results)}"

def fetch_robots(base: str) -> str:
    """Returnează conținutul fișierului /robots.txt sau None dacă lipsește."""
    try:
        res = requests.get(base + "/robots.txt", timeout=TIMEOUT)
        if res.status_code == 200:
            return res.text
    except requests.RequestException:
        pass
    return None

def disallowed_paths(robots_text: str) -> list:
    """Extrage și returnează o listă cu toate valorile Disallow dintr-un text
robots.txt."""
    paths = []
    if robots_text is None:
        return paths
    for line in robots_text.splitlines():
        line = line.strip()
        if line.lower().startswith("disallow:"):
            path = line.split(":", 1)[1].strip()
            if path:
                paths.append(path)
    return paths

def response_times(*urls: str) -> dict:
    """Măsoară timpul de răspuns pentru un număr variabil de URL-uri și
returnează {url: secunde}."""
    times = {}
    for url in urls:
        try:
            start = time.perf_counter()
            requests.get(url, timeout=TIMEOUT)
            times[url] = round(time.perf_counter() - start, 4)
        except requests.RequestException:
            times[url] = None
        time.sleep(1)
    return times

def log(message: str, **details):
    """Afișează un mesaj de jurnalizare urmat de detalii transmise sub formă de
cheie=valoare."""
```

```
    details_str = " | ".join(f"{k}={v}" for k, v in details.items())
    print(f"{message} | {details_str}" if details_str else message)
```

Anexa D. Codul fișierului part4_modules.py

```
# Laborator: funcții, metode și importuri pe web
# Student: Adrian Railean

import requests
import time
import urllib.parse
import re
from html.parser import HTMLParser
import hashlib
import json
import socket
import ssl
from datetime import datetime

BASE_URL = "https://cybercor.org"
TIMEOUT = 10  # secunde

def extract_links(html: str) -> list:
    """Extrage toate legăturile (href) dintr-un șir HTML, eliminând
duplicatele."""
    links = re.findall(r'href="([^"]+)"', html)
    return list(set(links))

def split_links(links: list, domain: str) -> tuple:
    """Împarte lista de legături în două liste: interne (pe același domeniu) și
externe."""
    internal, external = [], []
    for link in links:
        absolute_url = urllib.parse.urljoin(domain, link)
        parsed = urllib.parse.urlparse(absolute_url)
        if parsed.netloc == urllib.parse.urlparse(domain).netloc:
            internal.append(absolute_url)
        else:
            external.append(absolute_url)
    return internal, external

class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag.lower() == "img":
            src_val = dict(attrs).get("src")
            if src_val:
                self.images.append(src_val)

def page_fingerprint(url: str) -> str:
    """Calculează amprenta SHA-256 a conținutului unei pagini web."""
    res = requests.get(url, timeout=TIMEOUT)
    return hashlib.sha256(res.content).hexdigest()

def resolve(hostname: str) -> str:
    """Returnează adresa IP asociată unui nume de domeniu."""
    return socket.gethostbyname(hostname)
```

```
def cert_days_left(hostname: str) -> int:
    """Calculează câte zile au mai rămas până la expirarea certificatului
SSL."""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
            cert = ssock.getpeercert()
            expire_timestamp = ssl.cert_time_to_seconds(cert["notAfter"])
            current_timestamp = datetime.utcnow().timestamp()
            return int((expire_timestamp - current_timestamp) / (24 * 3600))
```

Anexa E. Codul modulului webtools.py

```
# Laborator: funcții, metode și importuri pe web
# Student: Adrian Railean

import requests
import time
import json
import socket
import ssl
from datetime import datetime
import urllib.parse
import re

TIMEOUT = 10  # secunde
DEFAULT_HEADERS = {"User-Agent": "WebLab-AdrianRailean"}

def fetch(url: str, timeout: int = TIMEOUT):
    """Descarcă o pagină web folosind antete implicite și returnează obiectul
răspuns."""
    return requests.get(url, headers=DEFAULT_HEADERS, timeout=timeout)

def get_status(url: str) -> int:
    """Returnează codul de stare HTTP pentru adresa url."""
    response = requests.get(url, headers=DEFAULT_HEADERS, timeout=TIMEOUT)
    return response.status_code

def get_title(html: str) -> str:
    """Extrage și returnează titlul paginii dintr-un șir HTML."""
    start_tag, end_tag = "<title>", "</title>"
    start_idx = html.find(start_tag)
    if start_idx != -1:
        start_pos = start_idx + len(start_tag)
        end_pos = html.find(end_tag, start_pos)
        if end_pos != -1:
            return html[start_pos:end_pos].strip()
    return "Titlu negăsit"

def security_headers(url: str) -> dict:
    """Verifică prezența celor 5 antete de securitate și returnează un
dicționar {antet: True/False}."""
    headers_to_check = [
        "Strict-Transport-Security", "Content-Security-Policy",
        "X-Frame-Options", "X-Content-Type-Options", "Referrer-Policy"
    ]
    try:
        res = requests.get(url, headers=DEFAULT_HEADERS, timeout=TIMEOUT)
        return {h: h in res.headers for h in headers_to_check}
```

```
    except requests.RequestException:
        return {h: False for h in headers_to_check}

def score_headers(results: dict) -> str:
    """Calculează un scor sub formă de șir bazat pe dicționarul de antete."""
    return f"{sum(1 for val in results.values() if val)}/{len(results)}"

def resolve(hostname: str) -> str:
    """Returnează adresa IP a unui domeniu."""
    return socket.gethostbyname(hostname)

def cert_days_left(hostname: str) -> int:
    """Calculează zilele rămase până la expirarea certificatului SSL."""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
            cert = ssock.getpeercert()
            expire_timestamp = ssl.cert_time_to_seconds(cert["notAfter"])
            current_timestamp = datetime.utcnow().timestamp()
            return int((expire_timestamp - current_timestamp) / (24 * 3600))

def extract_links(html: str) -> list:
    """Extrage legăturile unice dintr-un șir HTML."""
    return list(set(re.findall(r'href="([^"]+)"', html)))

def split_links(links: list, domain: str) -> tuple:
    """Împarte legăturile în interne și externe."""
    internal, external = [], []
    for link in links:
        abs_url = urllib.parse.urljoin(domain, link)
        if urllib.parse.urlparse(abs_url).netloc ==
urllib.parse.urlparse(domain).netloc:
            internal.append(abs_url)
        else:
            external.append(abs_url)
    return internal, external

def fetch_robots(base: str) -> str:
    """Returnează conținutul fișierului /robots.txt sau None."""
    try:
        res = requests.get(base + "/robots.txt", headers=DEFAULT_HEADERS,
timeout=TIMEOUT)
        if res.status_code == 200:
            return res.text
    except requests.RequestException:
        pass
    return None

def disallowed_paths(robots_text: str) -> list:
    """Extrage căile interzise din robots.txt."""
    paths = []
    if robots_text is None:
        return paths
    for line in robots_text.splitlines():
        line = line.strip()
        if line.lower().startswith("disallow:"):
            path = line.split(":", 1)[1].strip()
            if path:
                paths.append(path)
    return paths
```

```
def site_report(url: str):
    """Generează raportul complet al site-ului, îl afișează și îl salvează în
report.json."""
    parsed_base = urllib.parse.urlparse(url)
    hostname = parsed_base.netloc

    res = fetch(url)
    title = get_title(res.text)

    try:
        ip_addr = resolve(hostname)
    except Exception:
        ip_addr = "N/A"

    history_str = "Niciuna"
    try:
        res_http = requests.get(f"http://{hostname}", headers=DEFAULT_HEADERS,
timeout=TIMEOUT)
        if res_http.history:
            history_str = " -> ".join([str(h.status_code) for h in
res_http.history]) + f" -> {res_http.url}"
    except Exception:
        pass

    sec_score = score_headers(security_headers(url))

    try:
        cert_str = f"{cert_days_left(hostname)} de zile rămase"
    except Exception:
        cert_str = "N/A"

    links = extract_links(res.text)
    internal, external = split_links(links, url)
    disallowed = disallowed_paths(fetch_robots(url))

    report_data = {
        "status_code": res.status_code,
        "final_url": res.url,
        "title": title,
        "ip_address": ip_addr,
        "redirect_history": history_str,
        "security_score": sec_score,
        "certificate": cert_str,
        "internal_links_count": len(internal),
        "external_links_count": len(external),
        "disallowed_paths": disallowed
    }

    with open("report.json", "w", encoding="utf-8") as f:
        json.dump(report_data, f, indent=2)

    print(f"=== Raport site: {url} ===")
    print(f"Cod de stare:      {res.status_code}")
    print(f"Titlu:             {title}")
    print(f"Adresă IP:         {ip_addr}")
    print(f"Redirecționări:    {history_str}")
    print(f"Scor securitate:   {sec_score}")
    print(f"Certificat:        {cert_str}")
    print(f"Legături:          {len(internal)} interne, {len(external)}
```

```
externe")
    print(f"Căi interzise:     {', '.join(disallowed) if disallowed else
'Niciuna'}")
    print("Salvat în report.json")

if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Anexa F. Codul fișierului main.py

```
# Laborator: funcții, metode și importuri pe web
# Student: Adrian Railean

import argparse
import csv
from datetime import datetime
import webtools

def check_paths(base: str, paths: list) -> dict:
    """Verifică o listă de căi pe un domeniu și returnează un dicționar {path:
status_code}."""
    results = {}
    for path in paths:
        url = base + path
        try:
            res = webtools.fetch(url)
            results[path] = res.status_code
        except Exception:
            results[path] = None
    return results

def main():
    parser = argparse.ArgumentParser(description="Script de analiză web pentru
laborator.")
    parser.add_argument("url", nargs="?", default="https://cybercor.org",
help="URL-ul țintă pentru analiză")
    args = parser.parse_args()

    target_url = args.url
    print(f"Pornire analiză pentru: {target_url}\n")

    # Exercițiul 49: Generare raport CSV
    paths_to_check = ["/", "/robots.txt", "/sitemap.xml"]
    path_results = check_paths(target_url, paths_to_check)
    checked_at_time = datetime.now().isoformat()

    with open("report.csv", "w", newline="", encoding="utf-8") as csvfile:
        fieldnames = ["path", "status", "checked_at"]
        writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
        writer.writeheader()
        for path, status in path_results.items():
            writer.writerow({
                "path": path,
                "status": status if status is not None else "Error",
                "checked_at": checked_at_time
            })
    print("Raportul CSV a fost salvat cu succes în report.csv\n")

    # Exercițiul 50: Proiect final
    webtools.site_report(target_url)
```

```
if __name__ == "__main__":
    main()
```